

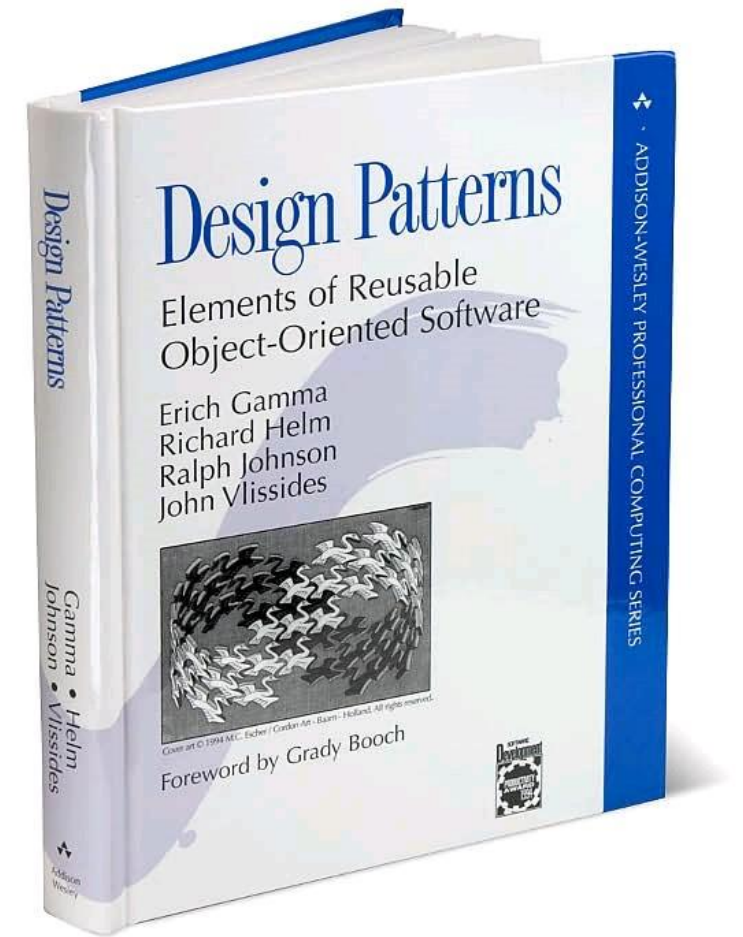
CSC3209 Software architecture and Design Patterns

Lecture13: Design Patterns Creational Patterns

Subject Lecturer: Dr. Muhammed Basheer Jasser

References

- Materials from
 - Gamma, Erich. Design patterns: elements of reusable object-oriented software. Pearson Education India, 1995.



Lecture Outline

- What is a Design Pattern
- Design Pattern Catalogue
 - Creational Patterns
 - Structural Patterns
 - Behavioural Patterns

Creational Patterns

- Creational design patterns abstract the instantiation process.
- They help make a system independent of how its objects are created, composed, and represented.

Creational Patterns

1. Abstract factory
2. Builder
- 3. Factory method**
4. Prototype
- 5. Singleton**

Factory : Creational Pattern

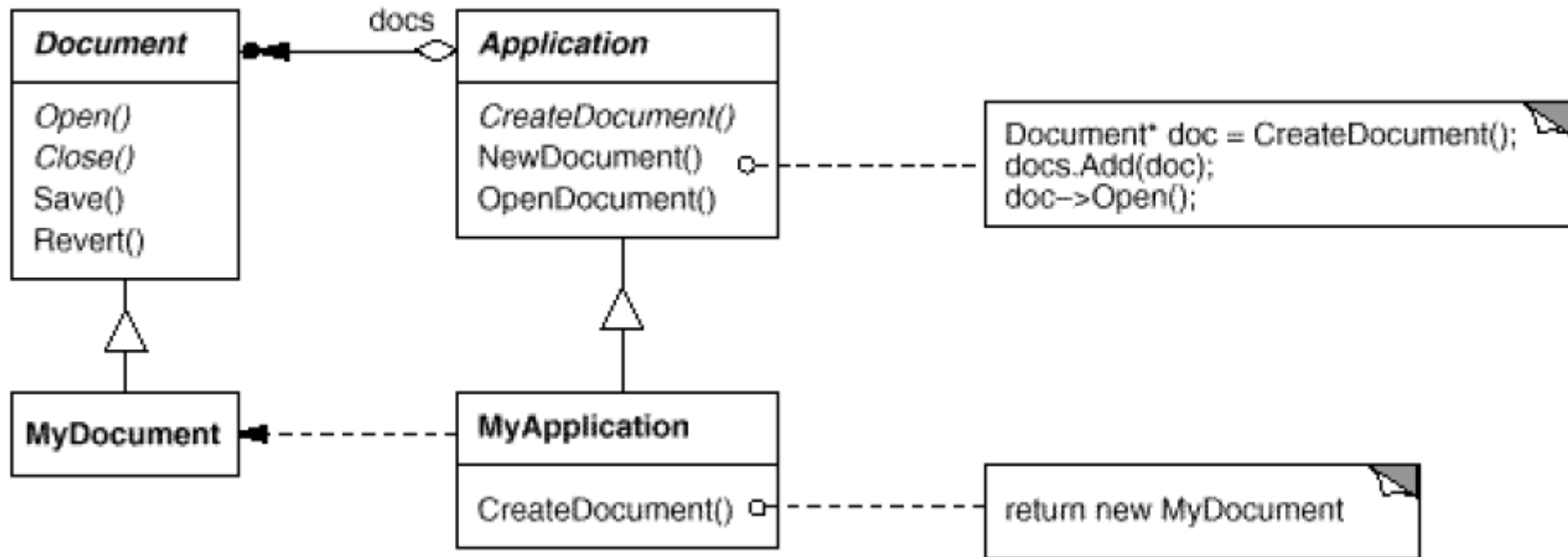
- **Intent:** Define an interface for creating an object, but let subclasses decide which class to instantiate. Factory Method lets a class defer instantiation to subclasses.
- **Also Known As:** Virtual Constructor
- **Motivation(1):**
 - Frameworks use abstract classes to define and maintain relationships between objects. A framework is often responsible for creating these objects as well.
 - Consider a framework for applications that can present multiple documents to the user. Two key abstractions in this framework are the classes Application and Document. Both classes are abstract, and clients have to subclass them to realize their application-specific implementations.
 - To create a drawing application, for example, we define the classes DrawingApplication and DrawingDocument. The Application class is responsible for managing Documents and will create them as required—when the user selects Open or New from a menu, for example.

Factory : Creational Pattern

- **Motivation(2):**
 - Because the particular Document subclass to instantiate is application-specific, the Application class can't predict the subclass of Document to instantiate—the Application class only knows when a new document should be created, not what kind of Document to create.
 - This creates a dilemma: The framework must instantiate classes, but it only knows about abstract classes, which it cannot instantiate.

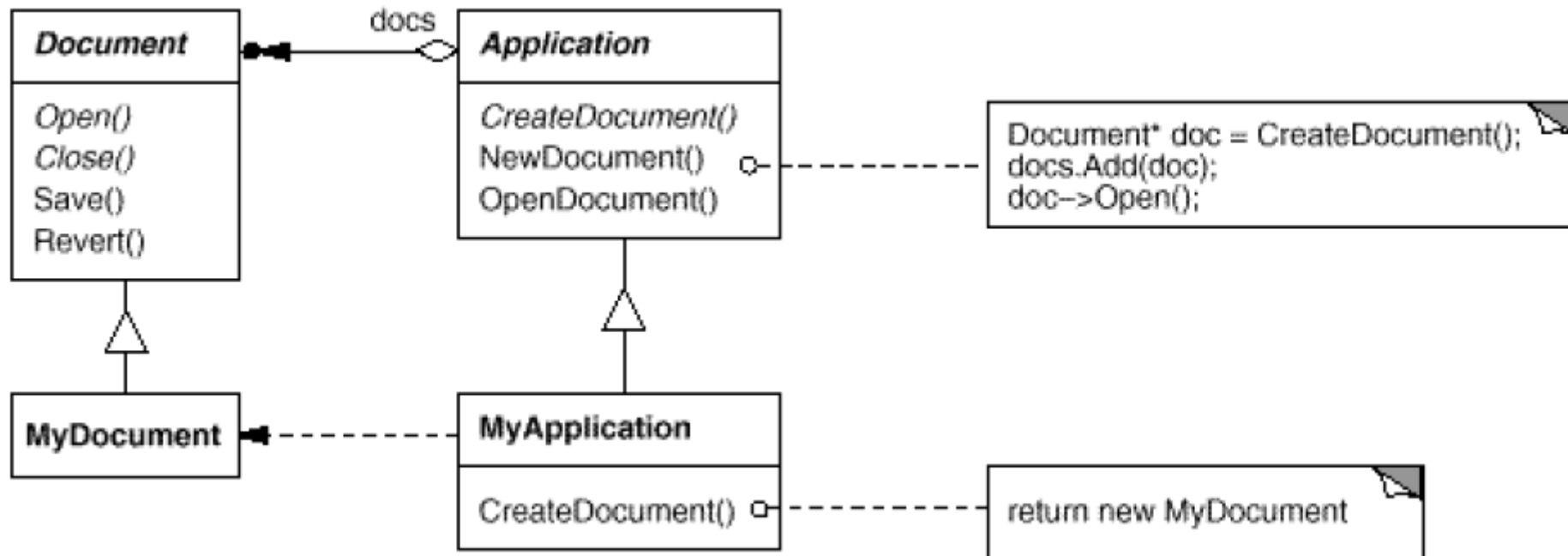
Factory : Creational Pattern

- **Motivation(3):**
 - The **Factory Method** pattern offers a solution. It **encapsulates** the **knowledge** of which **Document** subclass to **create** and **moves** this **knowledge** out of the **framework**.



Factory : Creational Pattern

- **Motivation(4):**
 - Application subclasses redefine an abstract CreateDocument operation on Application to return the appropriate Document subclass. Once an Application subclass is instantiated, it can then instantiate application-specific Documents without knowing their class. We call CreateDocument a factory method because it's responsible for "manufacturing" an object..

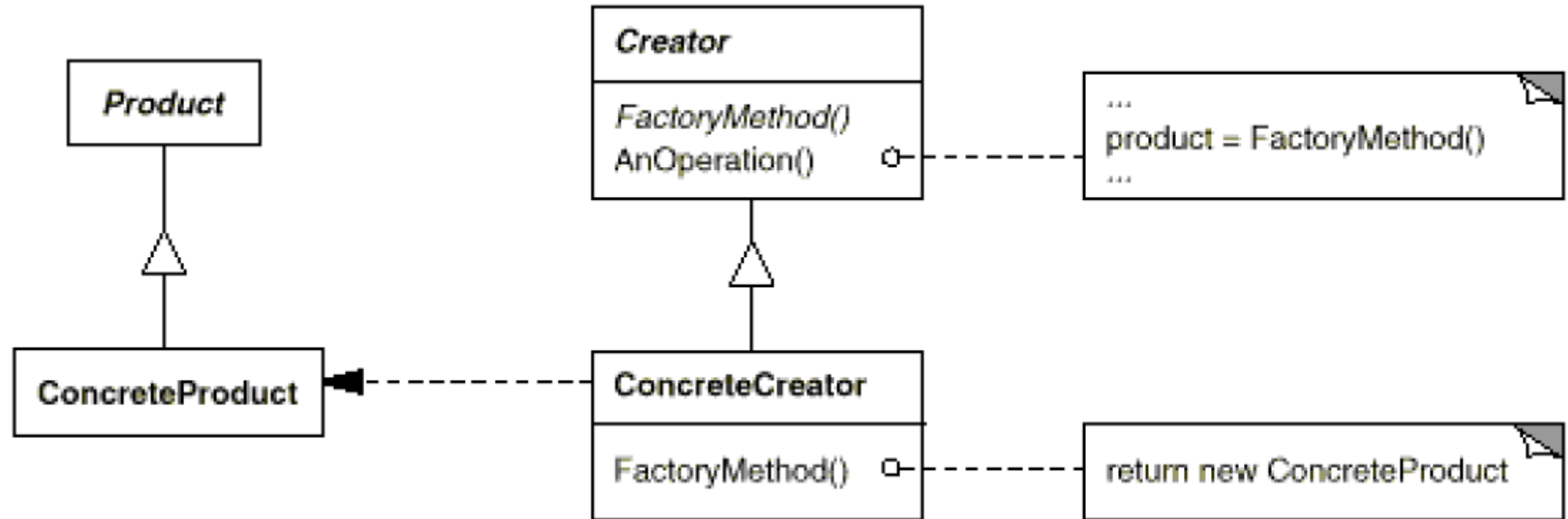


Factory : Creational Pattern

- **Applicability:** Use the Factory Method pattern when
 - a class can't anticipate the class of objects it must create.
 - a class wants its subclasses to specify the objects it creates.
 - classes delegate responsibility to one of several helper subclasses, and you want to localize the knowledge of which helper subclass is the delegate.

Factory : Creational Pattern

- Structure:



Factory : Creational Pattern

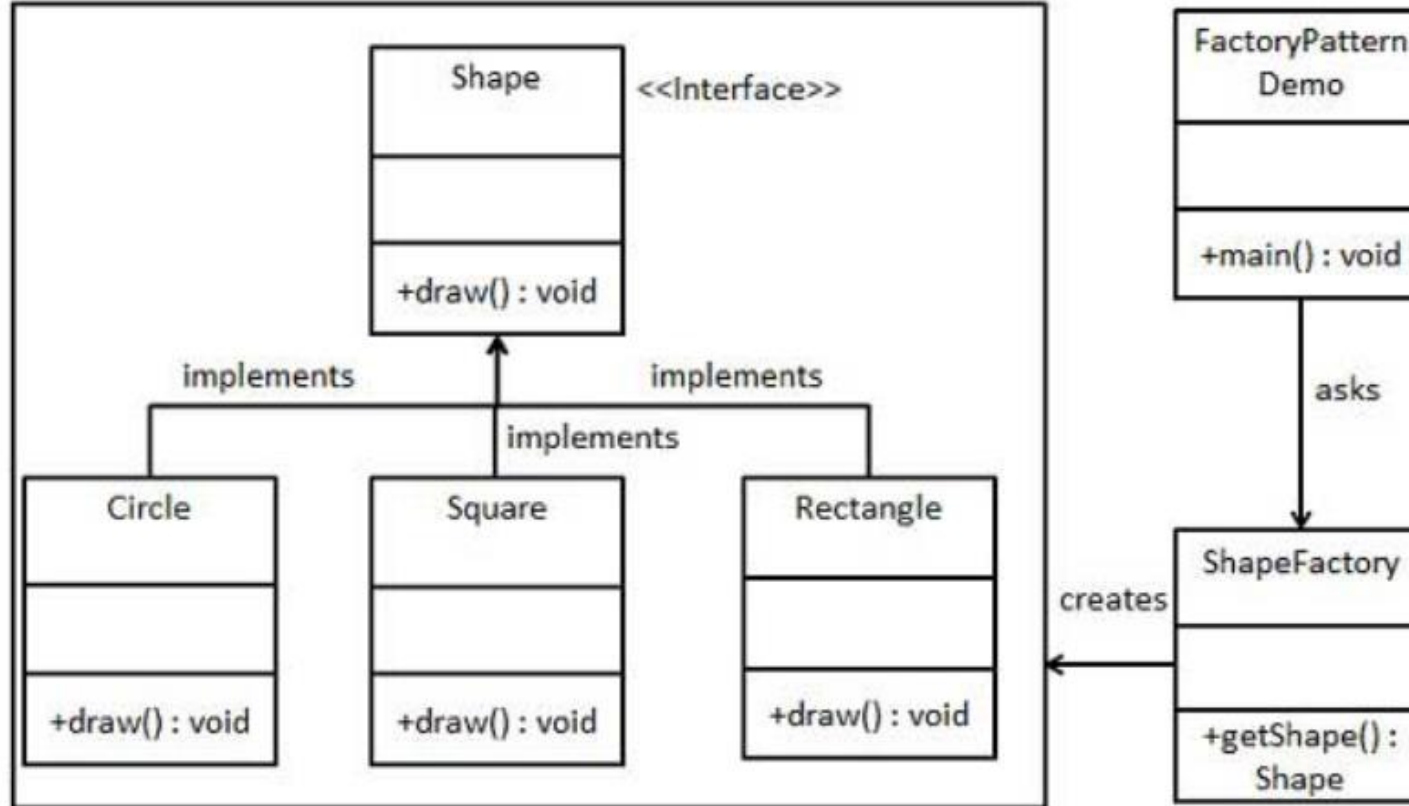
- **Participants:**
 - **Product** (Document)
 - defines the interface of objects the factory method creates.
 - **ConcreteProduct** (MyDocument)
 - implements the Product interface.
 - **Creator** (Application)
 - declares the factory method, which returns an object of type Product.
 - Creator may also define a default implementation of the factory method that returns a default ConcreteProduct object.
 - may call the factory method to create a Product object.
 - **ConcreteCreator** (MyApplication)
 - overrides the factory method to return an instance of a ConcreteProduct.

Factory : Creational Pattern

- **Collaborations:**
 - Creator relies on its subclasses to define the factory method so that it returns an instance of the appropriate ConcreteProduct.

Factory : Creational Pattern: Example

- We're going to create a Shape interface and concrete classes implementing the Shape interface. A factory class ShapeFactory is defined as a next step.
- FactoryPatternDemo, our demo class will use ShapeFactory to get a Shape object. It will pass information (CIRCLE / RECTANGLE / SQUARE) to ShapeFactory to get the type of object it needs.



Factory : Creational Pattern: Example

Create an interface.

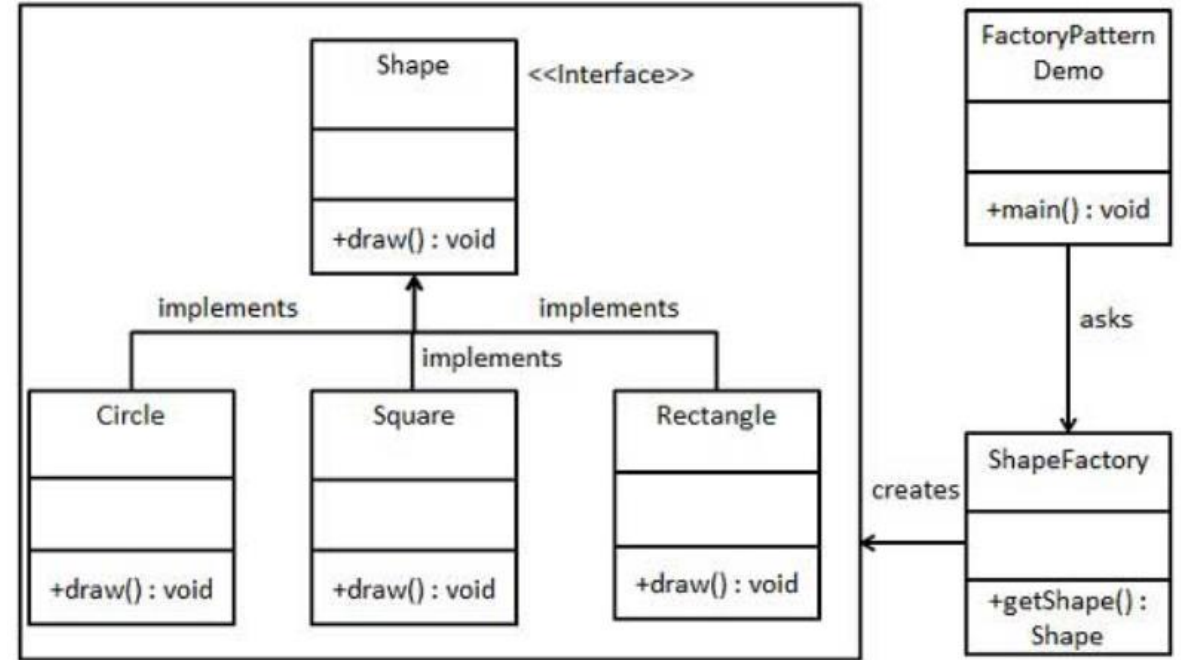
Shape.java

```
public interface Shape {  
    void draw();  
}
```

Create concrete classes implementing the same interface.

Rectangle.java

```
public class Rectangle implements Shape {  
  
    @Override  
    public void draw() {  
        System.out.println("Inside Rectangle::draw() method.");  
    }  
}
```



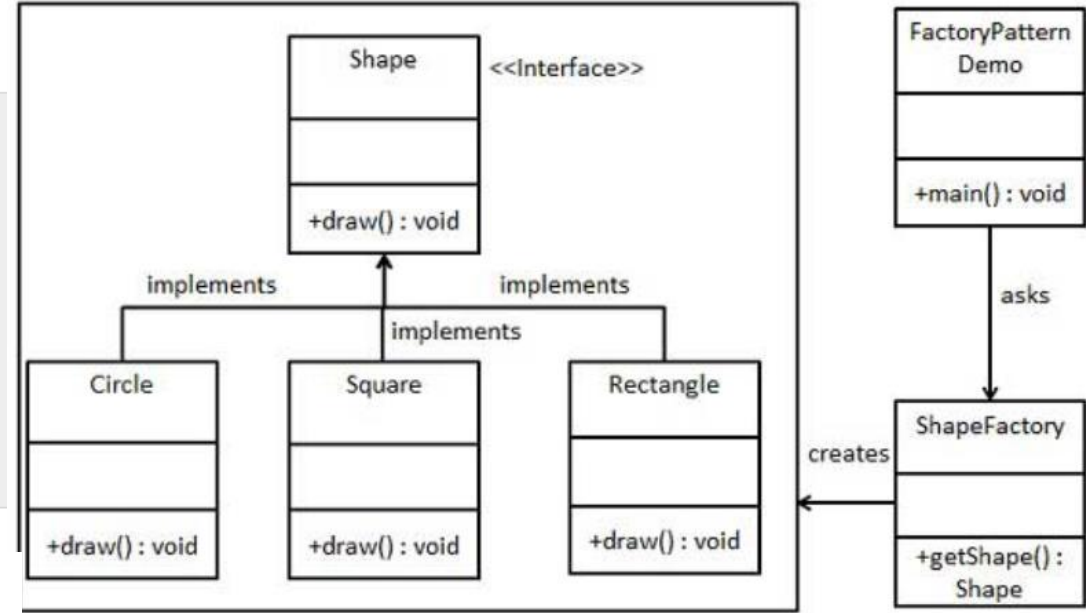
Factory : Creational Pattern: Example

Square.java

```
public class Square implements Shape {  
  
    @Override  
    public void draw() {  
        System.out.println("Inside Square::draw() method.");  
    }  
}
```

Circle.java

```
public class Circle implements Shape {  
  
    @Override  
    public void draw() {  
        System.out.println("Inside Circle::draw() method.");  
    }  
}
```

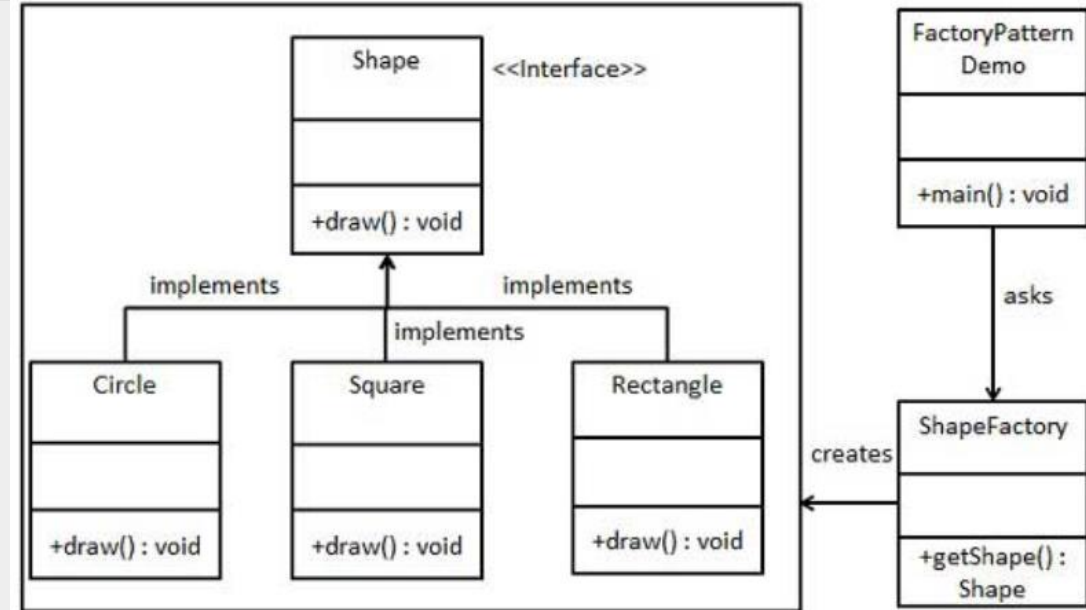


Factory : Creational Pattern: Example

Create a Factory to generate object of concrete class based on given information.

ShapeFactory.java

```
public class ShapeFactory {  
  
    //use getShape method to get object of type shape  
    public Shape getShape(String shapeType) {  
        if(shapeType == null){  
            return null;  
        }  
        if(shapeType.equalsIgnoreCase("CIRCLE")) {  
            return new Circle();  
        }  
        else if(shapeType.equalsIgnoreCase("RECTANGLE")) {  
            return new Rectangle();  
        }  
        else if(shapeType.equalsIgnoreCase("SQUARE")) {  
            return new Square();  
        }  
  
        return null;  
    }  
}
```

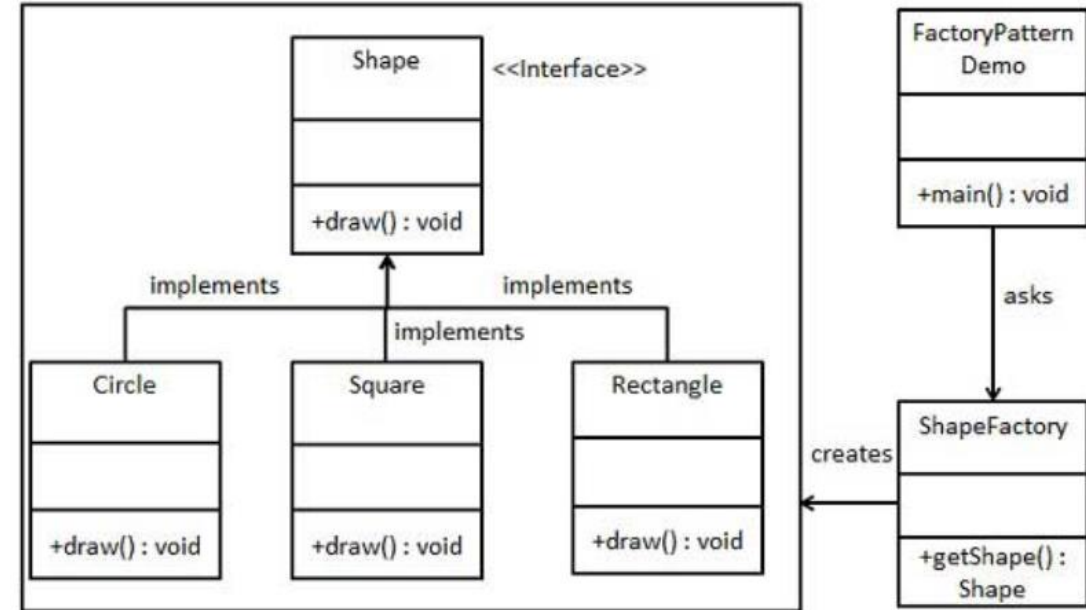


Factory : Creational Pattern: Example

Use the Factory to get object of concrete class by passing an information such as type.

FactoryPatternDemo.java

```
public class FactoryPatternDemo {  
  
    public static void main(String[] args) {  
        ShapeFactory shapeFactory = new ShapeFactory();  
  
        //get an object of Circle and call its draw method.  
        Shape shape1 = shapeFactory.getShape("CIRCLE");  
  
        //call draw method of Circle  
        shape1.draw();  
  
        //get an object of Rectangle and call its draw method.  
        Shape shape2 = shapeFactory.getShape("RECTANGLE");  
  
        //call draw method of Rectangle  
        shape2.draw();  
  
        //get an object of Square and call its draw method.  
        Shape shape3 = shapeFactory.getShape("SQUARE");  
  
        //call draw method of square  
        shape3.draw();  
    }  
}
```



Verify the output.

```
Inside Circle::draw() method.  
Inside Rectangle::draw() method.  
Inside Square::draw() method.
```

Singleton : Creational Pattern

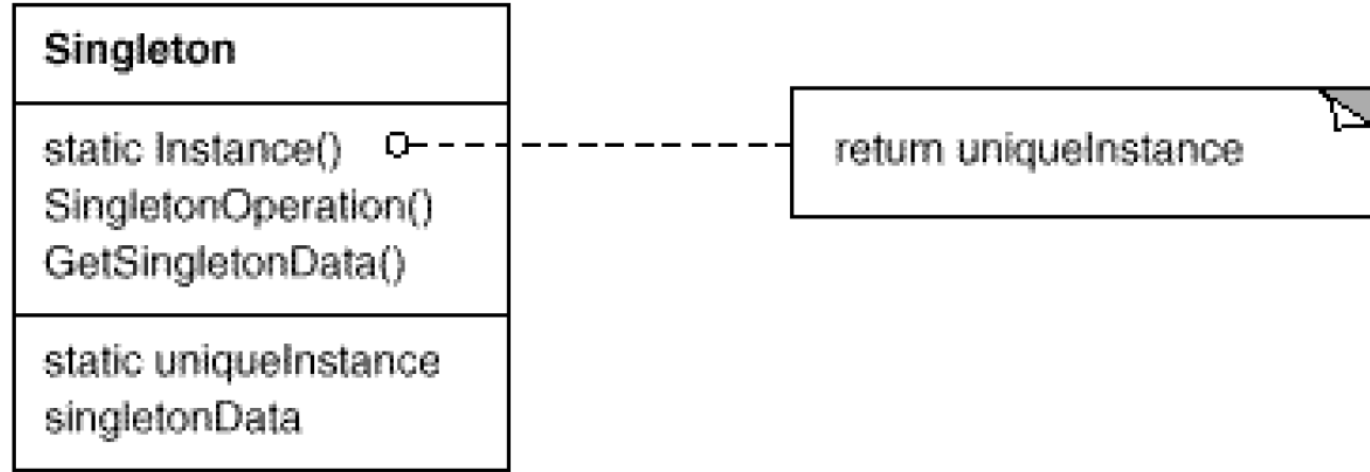
- **Intent:** Ensure a class only has one instance, and provide a global point of access to it.
- **Motivation:**
 - It's important for some classes to have exactly one instance. Although there can be many printers in a system, there should be only one printer spooler. There should be only one file system and one window manager. A digital filter will have one A/D converter. An accounting system will be dedicated to serving one company.
 - How do we ensure that a class has only one instance and that the instance is easily accessible? A global variable makes an object accessible, but it doesn't keep you from instantiating multiple objects.
 - A better solution is to make the class itself responsible for keeping track of its sole instance. The class can ensure that no other instance can be created (by intercepting requests to create new objects), and it can provide a way to access the instance. This is the Singleton pattern.

Singleton : Creational Pattern

- **Applicability:** Use the Singleton pattern when:
 - there **must be exactly one instance** of a class, and it must be accessible to clients from a well-known access point.
 - when the sole instance should be extensible by subclassing, and clients should be able to use an extended instance without modifying their code.

Singleton : Creational Pattern

- **Structure:**



- **Participants:**

- Singleton:

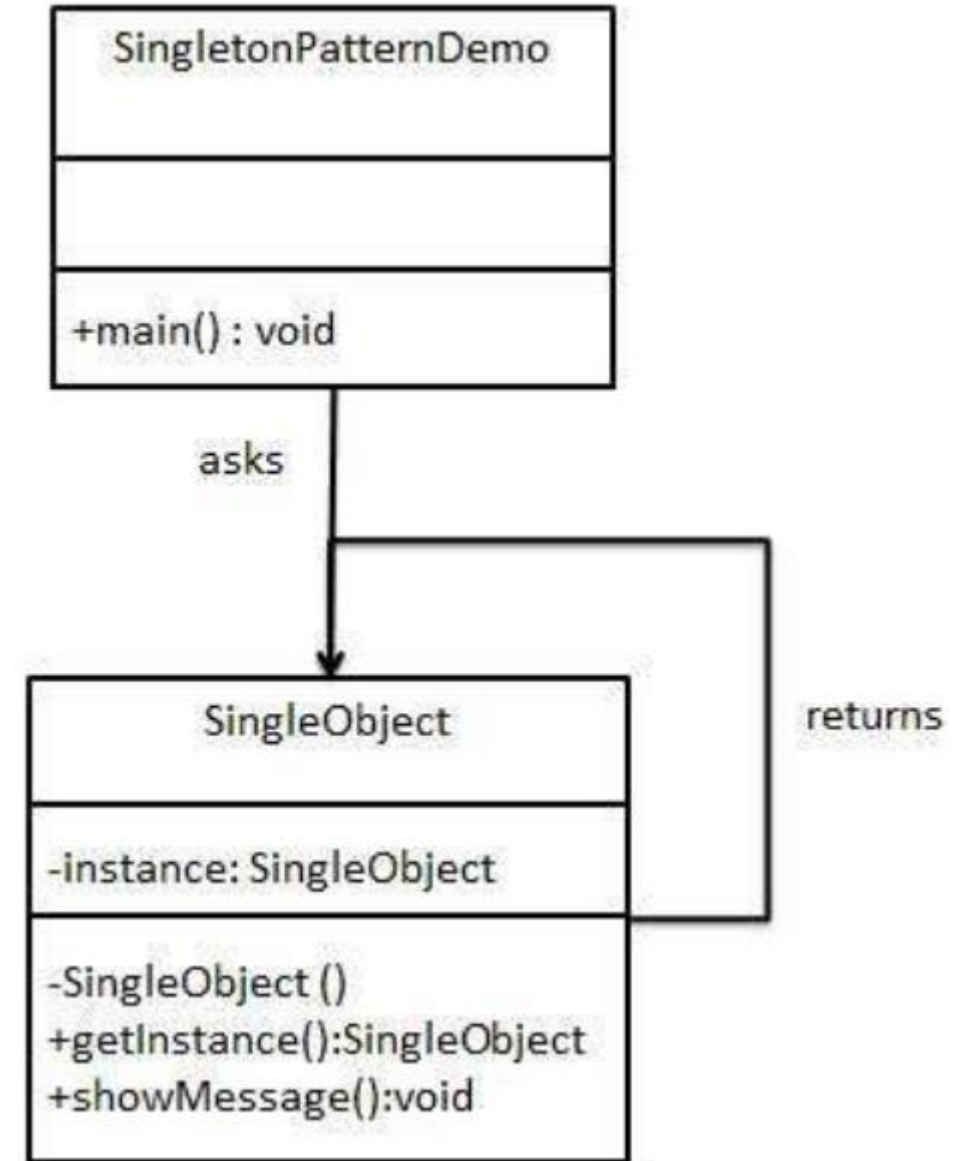
- defines an Instance operation that lets clients access its unique instance. Instance is a class operation (that is, a class method).
 - may be responsible for creating its own unique instance.

- **Collaborations:**

- Clients access a Singleton instance solely through Singleton's Instance operation

Singleton : Creational Pattern: Example

- We're going to **create** a **SingleObject** class. SingleObject class **have its constructor as private** and have a **static instance** of itself.
- **SingleObject** class **provides** a **static method** to **get its static instance** to outside world. SingletonPatternDemo, our demo class will **use SingleObject** class to **get** a **SingleObject** object.

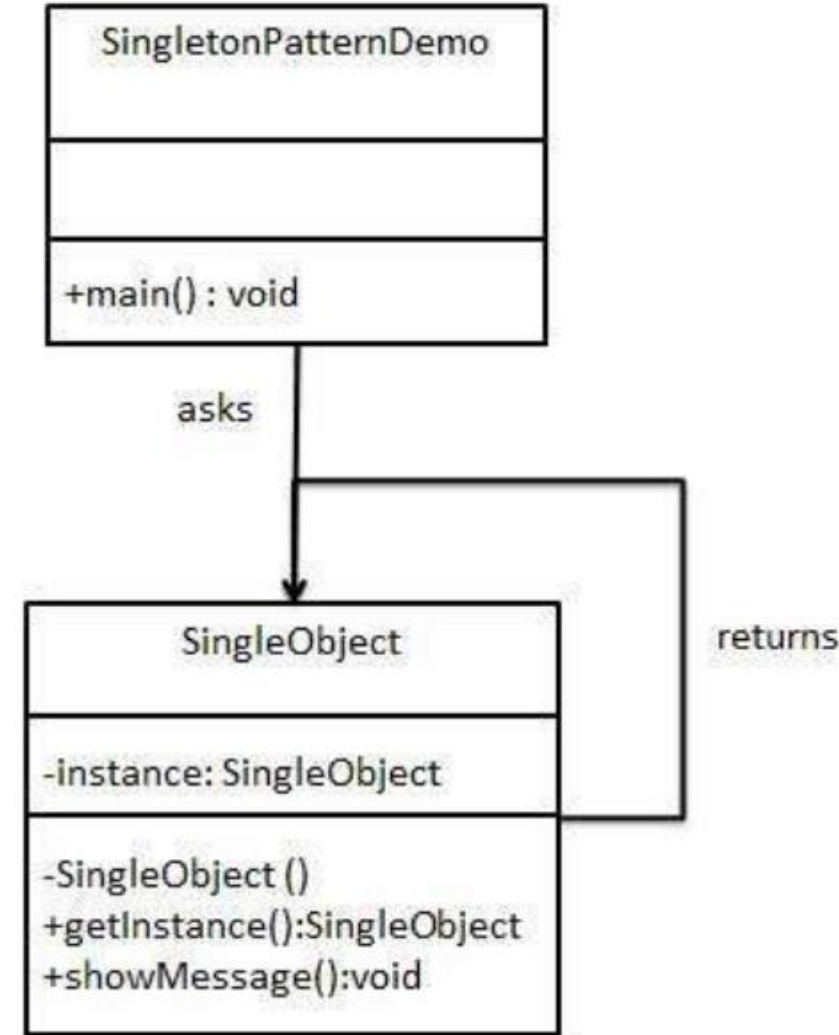


Singleton : Creational Pattern: Example

Create a Singleton Class.

SingleObject.java

```
public class SingleObject {  
  
    //create an object of SingleObject  
    private static SingleObject instance = new SingleObject();  
  
    //make the constructor private so that this class cannot be  
    //instantiated  
    private SingleObject() {}  
  
    //Get the only object available  
    public static SingleObject getInstance() {  
        return instance;  
    }  
  
    public void showMessage() {  
        System.out.println("Hello World!");  
    }  
}
```



Singleton : Creational Pattern: Example

Get the only object from the singleton class.

SingletonPatternDemo.java

```
public class SingletonPatternDemo {  
    public static void main(String[] args) {  
  
        //illegal construct  
        //Compile Time Error: The constructor SingleObject() is not visible  
        //SingleObject object = new SingleObject();  
  
        //Get the only object available  
        SingleObject object = SingleObject.getInstance();  
  
        //show the message  
        object.showMessage();  
    }  
}
```

Verify the output.

Hello World!

